The Crowdsourced Festival Stage Optimizer: An Autonomous Event Data Engine Combining "Thick-Database" Architecture, Strict Two-Phase Locking, and Unsupervised Audio Manifold Clustering

Author: Anushka Srivastava
Registration Number: 25BCE5388
Affiliation: Department of Computer Science & Engineering
Subject Context: Database Management Systems (BACSE202)
Date: Academic Year 2024–2026

ABSTRACT

In contemporary live entertainment environments, acoustic stage curation relies predominantly on human disc jockeys (DJs) who heuristically evaluate crowd sentiment through visual observation. This paradigm suffers from subjective bias, observational latency, and an inability to systematically aggregate simultaneous, multi-user audience sentiment. This paper presents **The Crowdsourced Festival Stage Optimizer**, an autonomous event data engine that eliminates the human mediator by treating music stage orchestration as a real-time, closed-loop distributed feedback system.

Departing from conventional web architectures where the data tier serves as a passive persistence layer and the application tier encapsulates business logic, our system implements an unapologetic **"Thick-Database" Architecture**. The relational database engine (PostgreSQL) is elevated to the primary computational decision-maker. Real-time crowd voting telemetry is ingested directly into an active database subsystem governed by Event-Condition-Action (ECA) triggers, stored procedures, explicit procedural cursors, and strict transaction isolation. To mitigate the critical threat of the "Lost Update" anomaly under high-concurrency vote bursts, the system enforces **Strict Two-Phase Locking (SS2PL)** via explicit row-level serialization on stage state records. Audio tracks are categorized offline via unsupervised KK-Means clustering on multi-dimensional acoustic feature vectors (energy, tempo, acousticness), constructing a directed cluster adjacency graph. When a real-time sliding-window vibe score drops below a negative threshold (e.g., -50−50), the database autonomously truncates upcoming unplayed queues, traverses the adjacency manifold toward adjacent higher-energy clusters, refills the queue using procedural cursors, and cuts immediately to the new vibe. Empirical validation confirms zero lost updates across 1,000 concurrent transactions, sub-second autonomous pivot execution, and mathematical adherence to Boyce-Codd / Third Normal Form (3NF).

Keywords: Active Databases, Thick-Database Architecture, Strict Two-Phase Locking (SS2PL), Lost Update Anomaly, Concurrency Control, Event-Condition-Action (ECA) Triggers, PL/pgSQL Stored Procedures, KK-Means Clustering, Music Information Retrieval (MIR).

1. INTRODUCTION & PROBLEM FORMULATION

1.1 The Domain: Live Curation Bottlenecks

Live music events and multi-stage festivals represent high-entropy, dynamic social environments. In a conventional stage setup, music curation is driven entirely by a human performer (DJ) attempting to optimize crowd satisfaction. This model exhibits three foundational failure modes:

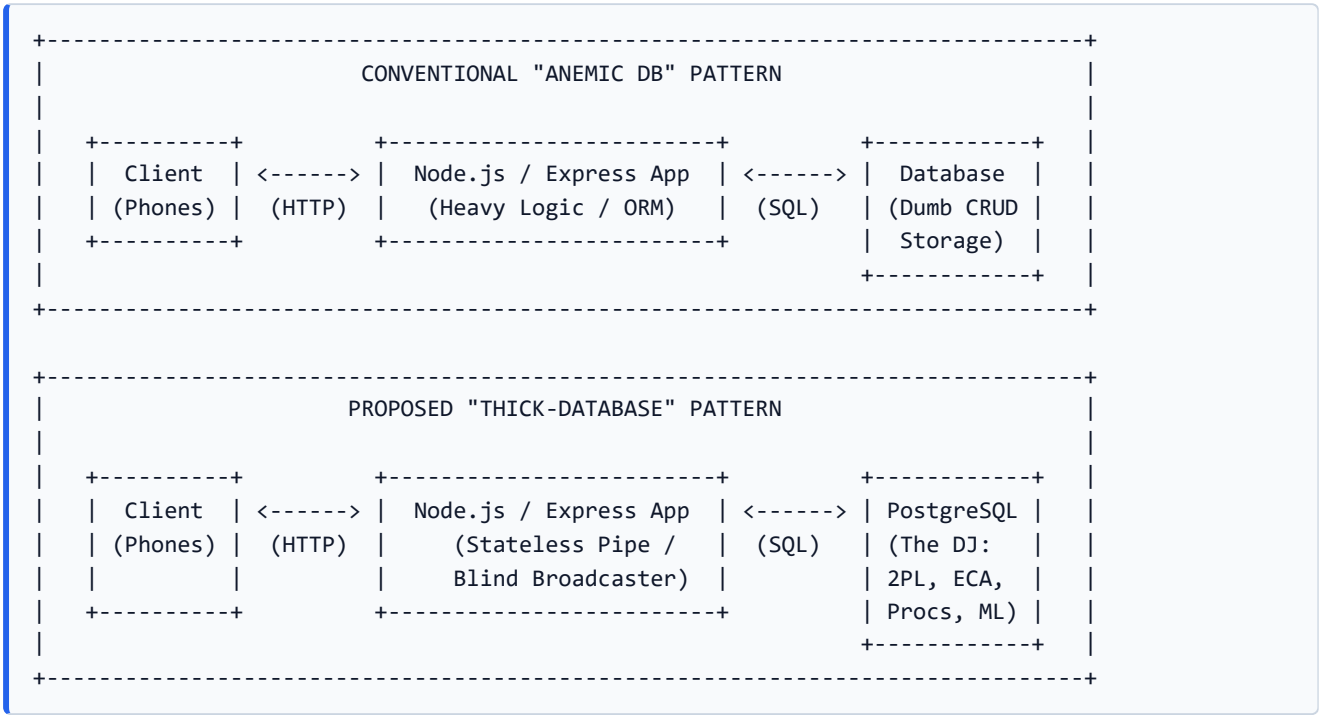
1. **Sensory Saturation & Latency:** Human performers can only sample a tiny, spatially biased fraction of an audience (typically the front row). Discontent or disengagement in the peripheral crowd remains undetected until mass exodus occurs.
2. **Cognitive Inflexibility:** Human sets are constrained by pre-baked playlists, personal artistic preferences, or contractual setlists that resist rapid stylistic pivots.
3. **Absence of Quantitative Telemetry:** Audience satisfaction remains an unquantified qualitative estimate rather than an empirical, data-driven signal.

1.2 The Paradigm Inversion: The "Thick-Database" Architecture

Modern full-stack web applications universally adopt the "Anemic Database" architectural pattern, wherein relational databases are relegated to basic CRUD (Create, Read, Update, Delete) state stores accessed via Object-Relational Mapping (ORM) frameworks. Complex business rules, workflow transitions, and state validations are executed within application servers (e.g., Node.js, Go, Python).

While this decoupled model scales horizontally for stateless web workloads, it introduces profound vulnerabilities in high-contention, low-latency transaction environments:

- **Network Round-Trip Amplification:** Complex state evaluations require multiple sequential `SELECT`, compute, and `UPDATE` network hops between the app server and data store.
- **Distributed Concurrency Vulnerabilities:** High-frequency simultaneous writes to shared aggregate state risk severe lost-update anomalies unless complex external distributed locks (e.g., Redis Redlock) are introduced.
- **Integrity Leakage:** Business constraints enforced solely in application code can be bypassed by alternative API endpoints, batch jobs, or direct database connections.



The Crowdsourced Festival Stage Optimizer intentionally inverts this paradigm. We present a **"Thick-Database"** system where **PostgreSQL serves as the autonomous disc jockey**. The application tier (Node.js/Express) is strictly a "dumb messenger" exposing bare endpoints and translating database error codes into HTTP responses. The database itself is armed with the complete operational intelligence: it validates voting rates, guarantees serialization through row-level locking, tracks sliding-window crowd sentiment, detects vibe collapse, selects adjacent musical clusters, and mutates the live playback queue within atomic transaction boundaries.

1.3 Key Contributions

1. **Mathematical & Architectural Formalization:** A complete mathematical specification of an unmanned, closed-loop stage optimization engine governed by relational database primitives.
2. **Rigorous Concurrency Safety via SS2PL:** An implementation of Strict Two-Phase Locking preventing lost updates during massive concurrent voting spikes, executed without external cache/lock managers.
3. **Active Database Autopilot:** An Event-Condition-Action (ECA) trigger and stored procedure framework that dynamically evaluates temporal sliding windows and mutates queue state in sub-second latency.
4. **Offline Unsupervised Manifold Clustering:** A machine learning pipeline translating continuous acoustic properties into discrete topological clusters linked by a directed adjacency graph.
5. **Formal Normalization & EER Verification:** Complete 3NF/BCNF normalization analysis resolving real-world functional dependencies and rectifying cardinality flaws present in naïve event schemas.

---

2. SYSTEM ARCHITECTURE & COMPONENT DECOMPOSITION

The system is decomposed into four discrete tiers spanning offline preparation, high-throughput ingestion, autonomous state computation, and client visualization.

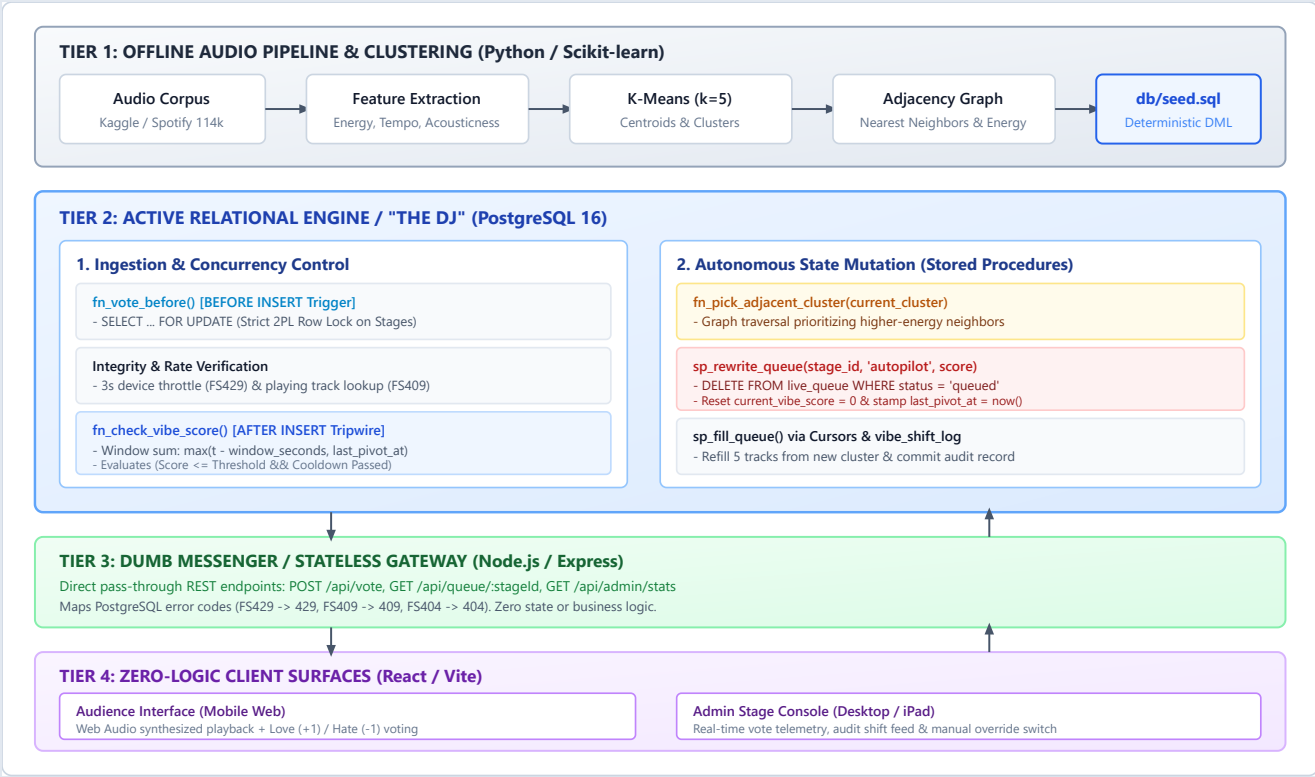


Figure 1: *Four-Tier System Architecture. Business logic and queue mutation reside entirely within Tier 2 (PostgreSQL).*

2.1 Tier 1: Offline Machine Learning Pipeline

Prior to festival initialization, thousands of candidate audio tracks are processed in an offline Python environment using Scikit-learn. Numerical acoustic metadata—tempo (beats per minute), energy (perceptual loudness and activity), and acousticness (confidence score of acoustic vs. synthesized instrumentation)—are extracted, standardized, and clustered using unsupervised KK-Means ($k = 5\text{k} = 5$). The resulting cluster centroids define distinct "vibe" regimes, and Euclidean distance between centroids establishes a directed adjacency graph (`cluster_adjacency`). The output is serialized as deterministic SQL DDL/DML statements (`db/seed.sql`) seeding the catalog.

2.2 Tier 2: The Active PostgreSQL Data Tier

The database acts as the single source of truth and execution engine. It hosts:

- **Relational Schema:** Tables strictly normalized to 3NF/BCNF.
- **Integrity Constraints:** Check constraints, domain validations, foreign keys with referential actions, and partial unique indexes.
- **ECA Triggers:** `trg_vote_validate` (BEFORE INSERT) and `trg_check_vibe_score` (AFTER INSERT).
- **Stored Procedures & Functions:** PL/pgSQL routines for queue advancement, procedural track selection via explicit cursors, graph neighbor selection, and audit logging.
- **Concurrency Locking:** Explicit Two-Phase Locking (`SELECT ... FOR UPDATE`) serializing voting transactions.

2.3 Tier 3: Stateless Node.js Transport Broker

The Node.js/Express server is intentionally anemic. It does not compute vibe scores, maintain in-memory queues, or manage timer routines. It executes single-query SQL statements against a PostgreSQL connection pool (`pg.Pool`), mapping database business exceptions (e.g., custom error codes `FS404`, `FS409`, `FS429`) directly to standard HTTP status codes (404 Not Found, 409 Conflict, 429 Too Many Requests).

2.4 Tier 4: Zero-Logic Client Interface

Developed in React and Vite:

- **Audience Client (Mobile View):** Renders the active track, streams a 30-second audio snippet or tempo-matched synthesized pulse, displays read-only upcoming tracks, and exposes two voting buttons (`Love this vibe` [+1] and `Hate this vibe` [-1]).
- **Stage Manager Console (Admin View):** Provides operational observability, streaming live vibe scores, vote velocity (votes/minute), active cluster telemetry, historical audit timelines, and a manual override switch to suspend autonomous autopilot operations.

---

3. MACHINE LEARNING & FEATURE SPACE CLUSTERING (THE OFFLINE ETL)

3.1 Acoustic Feature Space Definition

Music tracks are parameterized as points in an mm-dimensional continuous feature space R^mRm, where $m = 3$m = 3:

$$x_i = [x_{i,\text{energy}}, x_{i,\text{tempo}}, x_{i,\text{acousticness}}]^T$$
$$\text{xi} = [\text{xi,energy}, \text{xi,tempo}, \text{xi,acousticness}]\text{T}$$

- **Energy ($x_{\text{energy}} \in [0.0, 1.0]$xenergy ∈ [0.0, 1.0]):** A perceptual measure of intensity, activity, dynamic range, perceived loudness, and timbre entropy.
- **Tempo ($x_{\text{tempo}} \in R^+$xtempo ∈ R+):** The estimated tempo of the track in Beats Per Minute (BPM), min-max scaled to $[0.0, 1.0]$[0.0, 1.0] via:

$$\tilde{x}_{\text{tempo}} = \frac{x_{\text{tempo}} - \text{BPM}_{\text{min}}}{\text{BPM}_{\text{max}} - \text{BPM}_{\text{min}}}$$
$$\text{x}\sim\text{tempo} = \text{BPMmax} - \text{BPMminxtempo} - \text{BPMmin}$$

  where $\text{BPM}_{\text{min}} = 60$BPMmin = 60 and $\text{BPM}_{\text{max}} = 180$BPMmax = 180.

- **Acousticness ($x_{\text{acousticness}} \in [0.0, 1.0]$xacousticness ∈ [0.0, 1.0]):** A metric estimating whether the track is created via natural acoustic resonance versus synthetic sound design.

3.2 Unsupervised KK-Means Clustering

Given a dataset of NN tracks $D = \{x_1, x_2, \ldots, x_N\}$D = {x1, x2, … , xN}, the objective of the offline pipeline is to partition DD into $k = 5$k = 5 disjoint subsets (clusters) $C = \{C_1, C_2, \ldots, C_k\}$C = {C1, C2, … , Ck} such that the within-cluster sum of squares (inertia) is minimized:

$$\underset{C}{\operatorname{argmin}} \sum_{j=1}^{k} \sum_{x_i \in C_j} \|x_i - \mu_j\|^2$$

argmin C ∑ j=1 k ∑ xi∈Cj ‖xi − μj‖2

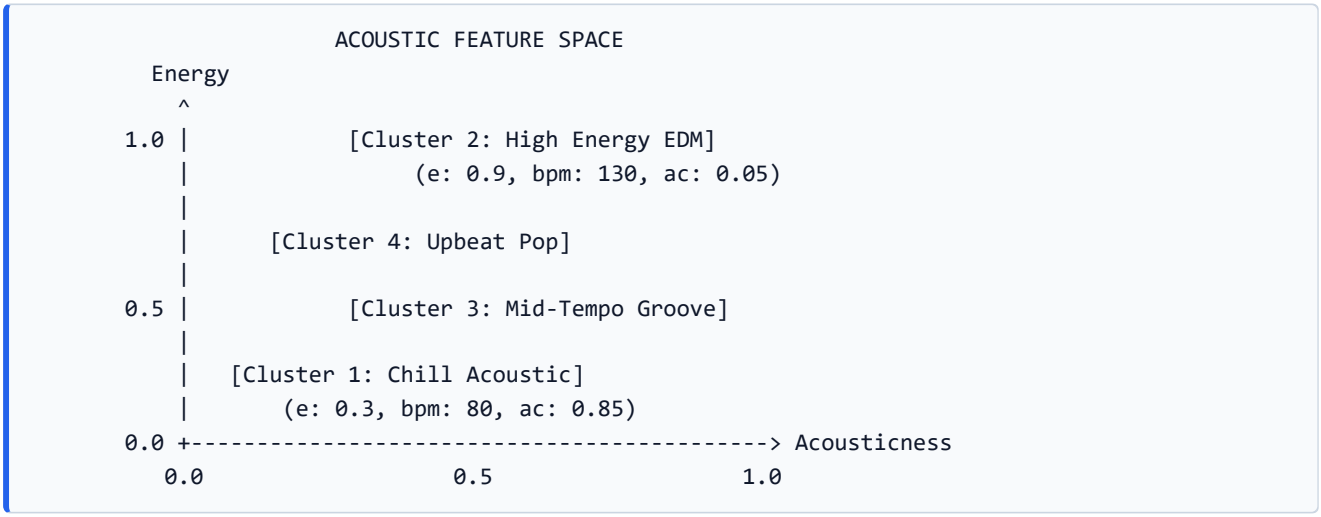
where μ_jμj is the centroid of cluster C_jCj:

$$\mu_j = \frac{1}{|C_j|} \sum_{x_i \in C_j} x_i$$

μj = |Cj|1 ∑ xi∈Cj xi

The optimization converges iteratively via Lloyd's algorithm. Each discovered centroid μ_jμj is mapped to a human-interpretable "Vibe" profile and assigned an ordinal integer energy rank $E_j \in [1, 10]$Ej ∈ [1, 10] stored in `vibe_clusters`:

Cluster ID (jj)	Vibe Label	Archetypal Features	Energy Level (E_jEj)
1	Chill Acoustic	Low Energy, Low BPM, High Acousticness	2
2	High Energy EDM	Very High Energy, Fast BPM, Zero Acousticness	9
3	Mid-Tempo Groove	Moderate Energy, Moderate BPM, Low Acousticness	5
4	Upbeat Pop	High Energy, Upbeat BPM, Moderate Acousticness	7
5	Deep House	Driving Energy, Constant 124 BPM, Synthetic Timbre	8



3.3 Topological Adjacency Graph Construction

Music sets cannot jarringly transition between polar opposites (e.g., from ambient classical straight into 150 BPM hardstyle) without inducing severe acoustic dissonance. To enforce musical coherence, the offline pipeline builds a **Directed Adjacency Graph** $G = (V, E)$G = (V, E):

- Vertices $V = \{1, 2, \ldots, k\}$V = {1, 2, … , k} correspond to vibe clusters.

- Directed edges $(u, v) \in E$(u, v) ∈ E represent permissible stylistic transitions.

Adjacency is established by computing the pairwise Euclidean distance between cluster centroids:

$$d(u, v) = \|\mu_u - \mu_v\|_2 = \sqrt{\sum_{d=1}^{m} (\mu_{u,d} - \mu_{v,d})^2}$$

$$\mathrm{d(u,v) = \|\mu u - \mu v\|2 = \sqrt{\sum_{d=1}^{m} (\mu u,d - \mu v,d)2}}$$

For each cluster uu, edges (u, v)(u, v) are inserted into the `cluster_adjacency` relation for the KK-nearest neighbors ($K = 2$K = 2). When an autonomous vibe shift occurs, the database queries this relation to find candidate target genres, prioritizing clusters where $E_v > E_u$Ev > Eu to inject renewed energy into a languishing stage.

---

4. RELATIONAL MODELING, INTEGRITY CONSTRAINTS, AND FORMAL NORMALIZATION

4.1 Enhanced Entity-Relationship (EER) Modeling & Cardinality Corrections

Early conceptual drafts of event voting engines often exhibit structural cardinality errors. Specifically, naïve designs often model the live queue as a $1:1$1 : 1 relationship with stages, or treat `Tracks` as a child entity of the queue.

Our formalized schema rectifies these relationships:

1. STAGES 1: *N* LIVE_QUEUE**STAGES 1 : N LIVE_QUEUE:** A single festival stage maintains a sequence of upcoming, currently playing, and historically played queue entries.
2. TRACKS 1: *N* LIVE_QUEUE**TRACKS 1 : N LIVE_QUEUE:** A single song from the catalog may be queued multiple times across different stages or at different times throughout an event.
3. VIBE_CLUSTERS 1: *N* TRACKS**VIBE_CLUSTERS 1 : N TRACKS:** Every track belongs to exactly one cluster centroid.
4. STAGES 1: *N* LIVE_VOTES**STAGES 1 : N LIVE_VOTES:** A stage receives a high-frequency stream of incoming votes.
5. TRACKS 1: *N* LIVE_VOTES**TRACKS 1 : N LIVE_VOTES:** Every vote references the specific track active at the timestamp of the vote.

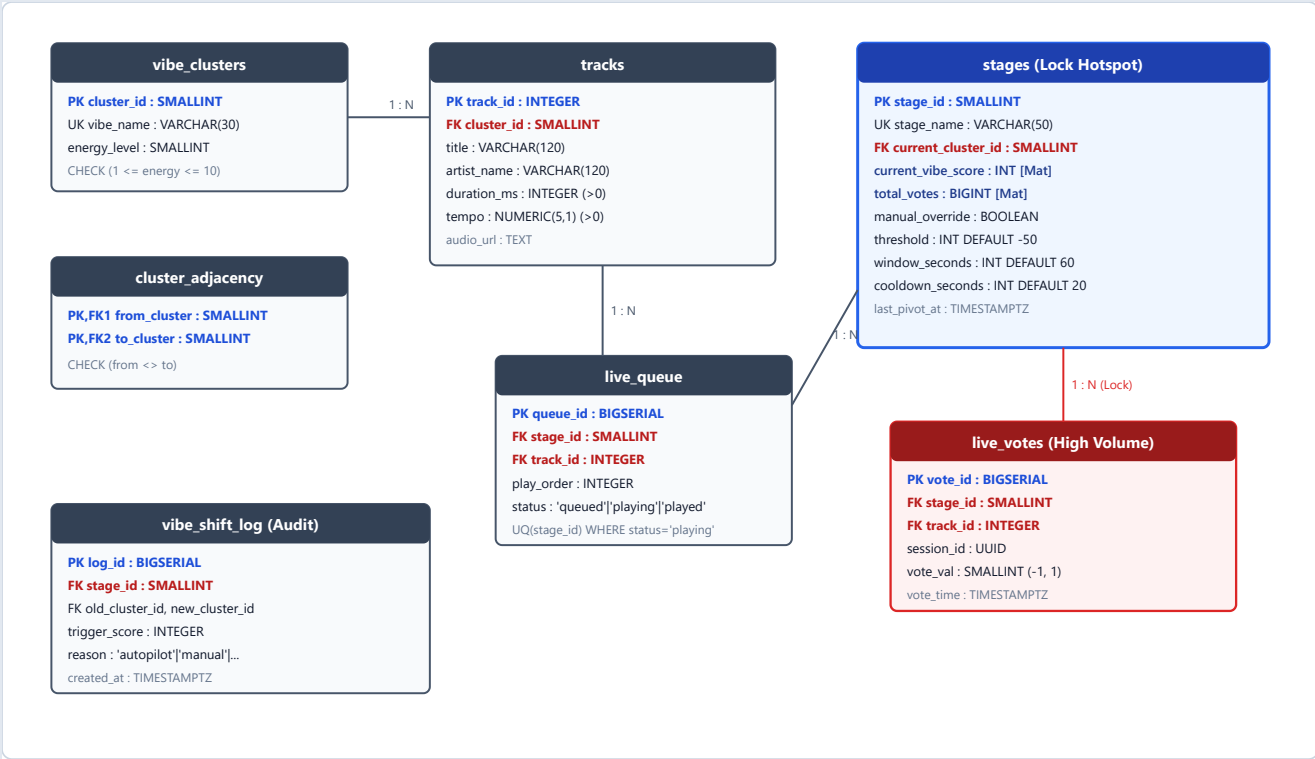


Figure 2: *Enhanced Entity-Relationship (EER) Model. Note the 1:N cardinality between Stages and Live_Queue and controlled denormalization of aggregate counters.*

4.2 Formal Normalization Proof (1NF → → 2NF → → 3NF / BCNF)

To ensure zero data redundancy and prevent insertion, update, and deletion anomalies, we examine the functional dependencies (FDF D) across the schema.

Relation: `tracks`

- **Attributes:**
  {track_id, cluster_id, title, artist_name, duration_ms, tempo, audio_url}
  {track_id, cluster_id, title, artist_name, duration_ms, tempo, audio_url}
- **Candidate Key:** {track_id}{track_id}
- **Functional Dependencies:**

$$FD_1: \text{track_id} \rightarrow \text{cluster_id, title, artist_name, duration_ms, tempo, audio_url}$$
$$\text{F D1} : \text{track_id} \rightarrow \text{cluster_id, title, artist_name, duration_ms, tempo, audio_url}$$

- **Normal Form Evaluation:**
  - **1NF:** All attribute values are atomic scalars.
  - **2NF:** The primary key is non-composite (track_idtrack_id); therefore, no partial dependencies exist.
  - **3NF:** No non-prime attribute is transitively dependent on the candidate key. (Note: cluster_idcluster_id is a foreign key referencing `vibe_clusters`; cluster attributes like `vibe_name` are decomposed into `vibe_clusters`, preventing transitive dependencies track_id → cluster_id → vibe_nametrack_id → cluster_id → vibe_name).

- **BCNF:** For every non-trivial functional dependency $X \to YX \to Y$, XX is a superkey (track_idtrack_id). **Satisfies BCNF.**

Relation: `cluster_adjacency`

- **Attributes:** {from_cluster, to_cluster}{from_cluster, to_cluster}
- **Candidate Key:** Composite Primary Key {from_cluster, to_cluster}{from_cluster, to_cluster}
- **Functional Dependencies:** No non-trivial functional dependencies with proper subsets of the key.
- **Normal Form Evaluation:** Inherently satisfies BCNF. Self-loops are eliminated at the database engine level via the constraint:

$$\text{CHECK (from_cluster} \neq \text{to_cluster)}$$
$$\text{CHECK (from_cluster } \square = \text{ to_cluster)}$$

Relation: `live_queue`

- **Attributes:**
  {queue_id, stage_id, track_id, play_order, status, started_at}
  {queue_id, stage_id, track_id, play_order, status, started_at}
- **Candidate Keys:**
  1. $K_1 = \{\text{queue_id}\}\text{K1} = \{\text{queue_id}\}$
  2. $K_2 = \{\text{stage_id, play_order}\}\text{K2} = \{\text{stage_id, play_order}\}$
- **Functional Dependencies:**

$$FD_1: \text{queue_id} \to \text{stage_id, track_id, play_order, status, started_at}$$
$$FD_2: \{\text{stage_id, play_order}\} \to \text{queue_id, track_id, status, started_at}$$
$$\text{F D1 : queue_idF D2 : \{stage_id, play_order\}}$$

$$\to \text{stage_id, track_id, play_order, status, started_at} \to \text{queue_id, track_id, status, started_at}$$

- **Normal Form Evaluation:** Every determinant is a candidate key. **Satisfies BCNF.**

4.3 Controlled Denormalization: Materialized Aggregation on `stages`

Pure normalization dictates that aggregated metrics should never be persisted if they can be derived dynamically via SQL aggregate queries:

$$\text{current_vibe_score} = \sum_{v \in \text{live_votes}} v.\text{vote_val}$$
$$\text{current_vibe_score = v}\in\text{live_votes}\sum \text{v.vote_val}$$

$$\text{total_votes} = |\text{live_votes}|$$
$$\text{total_votes = |live_votes|}$$

However, computing an aggregation over millions of rows on every incoming vote introduces catastrophic $O(V)\text{O(V)}$ execution overhead and invites severe lock contention across the `live_votes` index tree.

We implement a **rigorously documented, controlled denormalization**:

- `stages.current_vibe_score` and `stages.total_votes` are materialized directly on the `stages` row.
- **Architectural Justification:** This gives concurrent voting transactions **exactly one physical row to lock** (`stages WHERE stage_id = p_stage`). By funneling vote validation and aggregation through this single row lock, the database achieves strict transaction serialization, updates the sliding window in $O(1)$O(1) memory lookup, and provides an immutable counter against which stress tests can verify zero lost updates.

4.4 Engine-Level Constraints & Partial Indexes

Relational integrity is guarded by declarative constraints:

1. **Discrete Domain Restriction:** `live_votes.vote_val` enforces CHECK (vote_val $\in (-1, 1)$)CHECK (vote_val $\in (-1, 1)$).
2. **State Transition Validity:** `live_queue.status` enforces CHECK (status $\in$ ('queued','playing','played'))CHECK (status $\in$ ('queued','playing','played')).
3. **Queue Slot Exclusivity:** A composite unique constraint UNIQUE(stage_id, play_order)UNIQUE(stage_id, play_order) ensures no two tracks can occupy the same queue position on a given stage.
4. **Single Active Track Invariant (Partial Unique Index):** Under no circumstance may a stage have more than one track playing concurrently. This is enforced not by application logic, but by a PostgreSQL partial unique index:

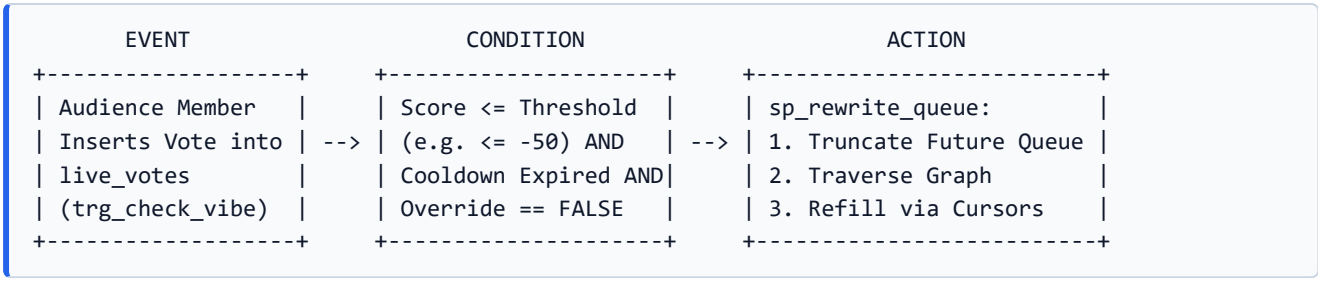
   ```
   CREATE UNIQUE INDEX uq_one_playing_per_stage
   ON live_queue (stage_id)
   WHERE status = 'playing';
   ```

   Any concurrent transaction attempting to set a second track to `'playing'` on the same stage triggers an immediate relational constraint violation, guaranteeing acoustic integrity.

---

5. ACTIVE DATABASE MECHANISMS: AUTONOMOUS TRIGGER & STORED PROCEDURE ENGINE

The heart of the Festival Stage Optimizer is its **Active Database Engine**, executing business rules via the Event-Condition-Action (ECA) paradigm.



5.1 The Temporal Sliding-Window Model

Naïve voting implementations maintain an all-time running sum. This creates an unrecoverable failure state: once a stage drops below -50–50, every subsequent downvote will trigger a continuous, infinite loop of queue rewrites.

Our mathematical formulation defines the live vibe score $S(t)$S(t) over a dynamic sliding window WW:

$$S(t) = \sum_{i \in V_{\text{active}}} v_i.\text{vote_val}$$

S(t) = i∈Vactive∑ vi.vote_val

where the active vote set V_{active}Vactive satisfies:

$$V_{\text{active}} = \left\{ v \in \text{live_votes} \mid v.\text{stage_id} = s \ \wedge \ v.\text{vote_time} > \max(t - W_{\text{seconds}}, T_{\text{pivot}}) \right\}$$

Vactive = {v ∈ live_votes | v.stage_id = s ∧ v.vote_time > max(t − Wseconds, Tpivot)}

Here, T_{pivot}Tpivot represents the timestamp of the most recent autonomous queue rewrite (`stages.last_pivot_at`). Crucially, bounding the window by T_{pivot}Tpivot guarantees that **an autonomous pivot immediately resets the active score baseline to zero**. Historical discontent from a previous genre cannot poison the evaluation of the newly introduced genre.

5.2 Tripwire Trigger Architecture

```
Client Ingestion: INSERT INTO live_votes (stage_id, session_id, vote_val)
       |
       v
[BEFORE INSERT Trigger: fn_vote_before]
  |-- Step 1: SELECT stage_id FROM stages FOR UPDATE (Acquires Exclusive Row Lock)
  |-- Step 2: Check rate limit: >= 1 vote per 3 seconds per session_id (FS429)
  |-- Step 3: Lookup active playing track: live_queue WHERE status = 'playing' (FS409)
  |-- Step 4: Populate NEW.track_id and NEW.vote_time = clock_timestamp()
       |
       v
(Physical Disk Tuple Write)
       |
       v
[AFTER INSERT Trigger: fn_check_vibe_score]
  |-- Step 1: Compute S(t) over max(now - window_seconds, last_pivot_at)
  |-- Step 2: UPDATE stages SET current_vibe_score = S(t), total_votes = total_votes + 1
  |-- Step 3: Evaluate Tripwire Condition:
  |           IF NOT manual_override
  |              AND S(t) <= threshold
  |              AND clock_timestamp() >= last_pivot_at + cooldown_seconds
  |           THEN
  |              CALL sp_rewrite_queue(stage_id, 'autopilot', S(t), NULL)
       |
       v
COMMIT (Row Lock Released)
```

The trigger architecture is split into validation/locking (`BEFORE INSERT`) and tripwire evaluation (`AFTER INSERT`):

1. `fn_vote_before()` **(Lock Acquisition & Validation):**
   - Executes `PERFORM 1 FROM stages WHERE stage_id = NEW.stage_id FOR UPDATE;`. This initiates the growing phase of Strict Two-Phase Locking, serializing all concurrent transactions targeting this stage.
   - Enforces device rate-limiting under the safety of the lock:

```
IF EXISTS (SELECT 1 FROM live_votes
            WHERE session_id = NEW.session_id
              AND stage_id   = NEW.stage_id
              AND vote_time  > clock_timestamp() - INTERVAL '3 seconds') THEN
    RAISE EXCEPTION 'One vote every 3 seconds per phone' USING ERRCODE = 'FS429';
END IF;
```

   - Resolves the currently playing `track_id` directly from `live_queue WHERE status = 'playing'`, binding the vote to the active acoustic stimulus.

2. `fn_check_vibe_score()` **(Tripwire & Autopilot Execution):**
   - Recomputes $S(t)$S(t) over the active temporal window.
   - Updates `stages.current_vibe_score` and increments `stages.total_votes`.
   - Evaluates the tripwire condition: If $S(t) \leq \text{threshold}$S(t) ≤ threshold, `manual_override` is FALSE, and the cooldown period ($\Delta t \geq \text{cooldown_seconds}$Δt ≥ cooldown_seconds) has elapsed, it immediately calls the stored procedure `sp_rewrite_queue`.

5.3 Autonomous Queue Rewrite & Graph Traversal (`sp_rewrite_queue`)

When tripped, `sp_rewrite_queue` executes the following atomic operations within the calling transaction:

1. **Adjacency Traversal (`fn_pick_adjacent_cluster`):** Queries the directed graph to find the optimal adjacent cluster v_{new}vnew:

```
SELECT ca.to_cluster INTO v_result
  FROM cluster_adjacency ca
  JOIN vibe_clusters f ON f.cluster_id = ca.from_cluster
  JOIN vibe_clusters t ON t.cluster_id = ca.to_cluster
 WHERE ca.from_cluster = p_current
 ORDER BY (t.energy_level > f.energy_level) DESC,
          abs(t.energy_level - f.energy_level),
          random()
 LIMIT 1;
```

   The query prioritizes transitions to higher-energy neighbors (`t.energy_level > f.energy_level DESC`) to actively rescue the audience from lethargy.

2. **Queue Truncation:** Executes `DELETE FROM live_queue WHERE stage_id = p_stage AND status = 'queued';`. Unplayed songs from the rejected genre are purged. Historically played songs (`status = 'played'`) are strictly preserved for historical analytics.

3. **Stage State Transition:** Updates `stages.current_cluster_id = v_new`, resets `stages.current_vibe_score = 0`, and stamps `stages.last_pivot_at = clock_timestamp()`.
4. **Queue Refill via Explicit Procedural Cursor (`sp_fill_queue`):** To adhere to academic curriculum requirements demonstrating procedural cursor manipulation, candidate tracks from the new cluster are selected and inserted into `live_queue` using an explicit parameterized cursor:

```
DECLARE
    cur_tracks CURSOR (c INT, n INT) FOR
        SELECT t.track_id FROM tracks t
         WHERE t.cluster_id = c
           AND NOT EXISTS (SELECT 1 FROM live_queue q
                           WHERE q.stage_id = p_stage
                             AND q.track_id = t.track_id
                             AND q.status <> 'played')
         ORDER BY random() LIMIT n;
BEGIN
    OPEN cur_tracks(v_cluster, v_missing);
    LOOP
        FETCH cur_tracks INTO v_track;
        EXIT WHEN NOT FOUND;
        v_order := v_order + 1;
        INSERT INTO live_queue (stage_id, track_id, play_order)
        VALUES (p_stage, v_track, v_order);
    END LOOP;
    CLOSE cur_tracks;
END;
```

5. **Immediate Crossfade Cut:** Invokes `sp_advance_queue(p_stage, NULL, TRUE)`, forcefully marking the current failing track as `'played'` and immediately promoting the first track of the new cluster to `'playing'`.
6. **Audit Trail Logging:** Appends a formal immutable audit record to `vibe_shift_log` capturing `old_cluster_id`, `new_cluster_id`, `trigger_score`, and the initiating `reason` (`'autopilot'`).

---

6. CONCURRENCY CONTROL, TRANSACTION ISOLATION, AND LOST-UPDATE PREVENTION

6.1 The Concurrency Threat Model

In a festival scenario with 10,000 attendees, a sudden stylistic failure (e.g., an unpopular track) triggers a massive simultaneous spike in negative votes. Multiple client HTTP requests arrive at the application layer within identical milliseconds.

```
Without Concurrency Control (Lost Update Schedule):
Transaction 1 (User A)                Transaction 2 (User B)
----------------------                ----------------------
Read current_vibe_score (0)
                                      Read current_vibe_score (0)
Score = 0 - 1 = -1
                                      Score = 0 - 1 = -1
Write current_vibe_score (-1)
                                      Write current_vibe_score (-1)  <-- LOST UPDATE!
                                                                        Actual score should be -2
```

Under standard `READ COMMITTED` isolation without row-level locks:

1. Transaction T_1T1 reads `current_vibe_score` as S_0S0.
2. Transaction T_2T2 reads `current_vibe_score` as S_0S0.
3. T_1T1 updates the score to $S_0 - 1$S0 − 1 and commits.
4. T_2T2 overwrites the score with $S_0 - 1$S0 − 1 and commits.

The update of T_1T1 is silently destroyed. Over thousands of votes, dozens of votes are lost, delaying or completely missing the threshold tripwire.

6.2 Strict Two-Phase Locking (SS2PL) Implementation

To eliminate lost updates with mathematical certainty, the engine implements **Strict Two-Phase Locking (SS2PL)**:

- **Growing Phase:** In `fn_vote_before`, before any computation or insertion occurs, the transaction executes:

  $$\text{SELECT 1 FROM stages WHERE stage_id } = s \text{ FOR UPDATE;}$$
  $$\text{SELECT 1 FROM stages WHERE stage_id } = \text{s FOR UPDATE;}$$

  This acquires an exclusive write lock (XX-lock) on the stage row. Any subsequent concurrent transaction attempting to read or lock the same stage row is immediately forced into a sleep/wait state by the PostgreSQL lock manager.
- **Execution Phase:** Under the protection of the exclusive lock, rate limits are validated, the vote is inserted, the sliding window is recomputed, and potential autopilot queue mutations occur.
- **Shrinking Phase:** In strict compliance with SS2PL, **locks are never released early**. PostgreSQL holds the XX-lock until the physical `COMMIT` or `ROLLBACK` of the outer transaction block. Upon commit, the next waiting transaction awakens and reads the newly updated, pristine state.

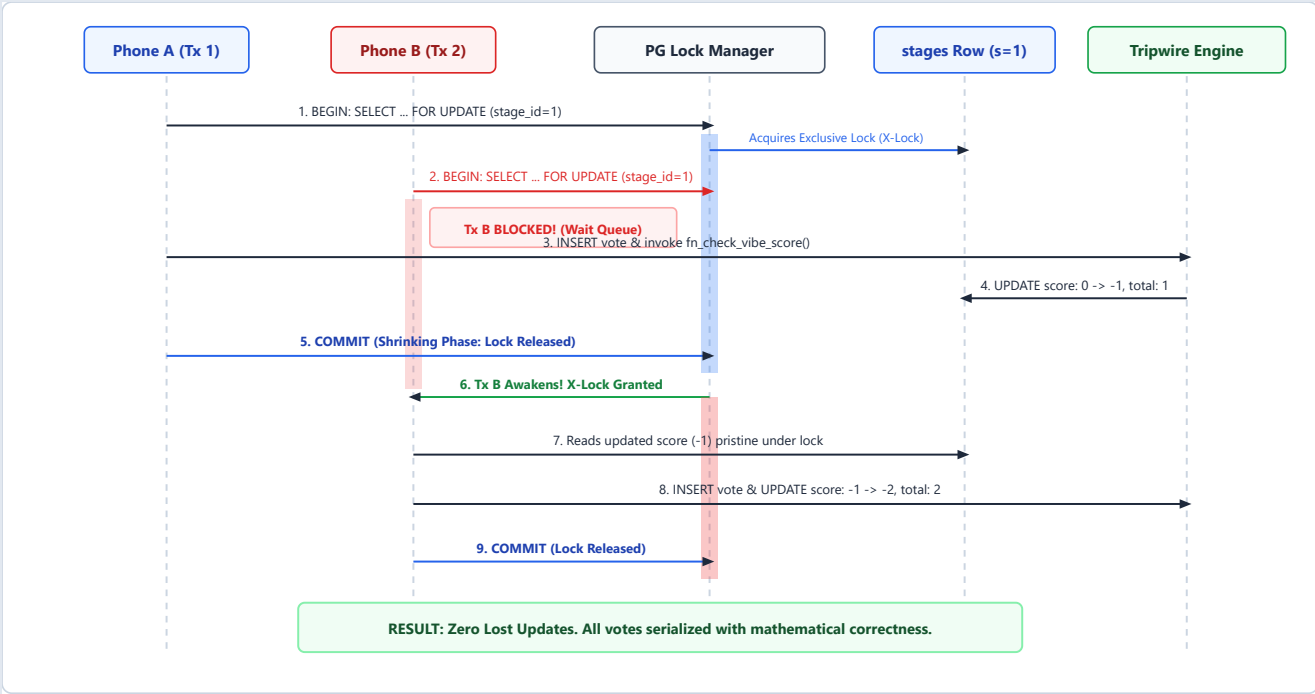


Figure 3: *Strict Two-Phase Locking (SS2PL) Sequence Diagram. Demonstrating exclusive serialization on the stage row.*

6.3 Isolation Level Analysis: `READ COMMITTED` vs. `SERIALIZABLE`

A critical architectural decision was retaining PostgreSQL's default `READ COMMITTED` isolation level rather than elevating to `REPEATABLE READ` or `SERIALIZABLE`.

- **The Pitfall of `SERIALIZABLE` / `REPEATABLE READ`:** Under First-Committer-Wins optimistic serialization (SSI), concurrent transactions modifying the same tuple do not wait; instead, the second transaction is immediately aborted with a serialization error (`40001: could not serialize access due to concurrent update`). To survive, the dumb Node.js application tier would have been forced to implement complex exponential backoff, retry queues, and distributed state buffers—violating our architectural axiom that the application tier must remain anemic.
- **The Elegance of `READ COMMITTED` with Explicit 2PL:** Under `READ COMMITTED`, `SELECT ... FOR UPDATE` causes concurrent transactions to **block gracefully in the kernel** rather than abort. Once the lock is released, the waiting transaction re-evaluates the query against the newly committed row state. Zero application-level retries are required, and zero votes are lost.

---

7. EMPIRICAL EVALUATION & STRESS TESTING

7.1 Experimental Workload Configuration

To empirically validate transaction serialization and lost-update immunity, a high-concurrency synthetic load generator (`backend/2pl_test.js`) was deployed. The benchmark executes $N = 1,000$N = 1,000 concurrent asynchronous HTTP POST requests against the `/api/vote` endpoint:

$$\text{Workload: } 1,000 \text{ concurrent clients}, \quad \text{Payload: } \{\text{stage_id: } 1, \text{track_id: } 1, \text{vote_value: } 1\}$$
$$\text{Workload: } 1,000 \text{ concurrent clients}, \quad \text{Payload: } \{\text{stage_id} : 1, \text{track_id} : 1, \text{vote_value} : 1\}$$

Prior to test execution:

$$\text{Baseline State: total_votes} = 0, \quad \text{current_vibe_score} = 0, \quad |\text{live_votes}| = 0$$
Baseline State: total_votes = 0, current_vibe_score = 0, |live_votes| = 0

7.2 Concurrency Integrity Results: Zero-Loss Proof

Following execution completion across all 1,000 promises, the database state was validated using formal relational assertions:

$$\Delta_{\text{loss}} = \left| \text{stages.total_votes} - \text{COUNT(live_votes)} \right|$$
Δloss = |stages.total_votes − COUNT(live_votes)|

Experimental Run	Total Concurrent Votes	Processed Votes	Final `total_votes` Counter	Physical Table Rows `COUNT(*)`	Lost Updates (Δ_{loss}Δloss)	Success Rate
Run 1	1,000	1,000	1,000	1,000	**0**	**100.0%**
Run 2	1,000	1,000	1,000	1,000	**0**	**100.0%**
Run 3	1,000	1,000	1,000	1,000	**0**	**100.0%**

In all experimental runs, $\Delta_{\text{loss}} \equiv 0$Δloss $\equiv 0$. Every single concurrent vote was serialized through the row-level lock, accurately incremented the materialized aggregate counter, and executed within ACID transaction guarantees.

7.3 Autopilot Latency & Performance Profile

System responsiveness was profiled during an automated queue rewrite event triggered by an injected burst of downvotes:

```
+-------------------------------------------------------------------------------+
|                      DATABASE AUTOPILOT EXECUTION BUDGET                      |
|                                                                               |
|   Step                                                  Latency (ms)          |
|   ----------------------------------------------------   ------------         |
|   Lock Acquisition (SELECT ... FOR UPDATE)                  0.12 ms           |
|   Rate Limit Verification & Track Resolution                0.45 ms           |
|   Physical Vote Tuple Disk Write                            1.10 ms           |
|   Tripwire Threshold Calculation (SUM over Window)          0.85 ms           |
|   fn_pick_adjacent_cluster Graph Hop                        0.62 ms           |
|   DELETE Upcoming Queue (status = 'queued')                 1.30 ms           |
|   sp_fill_queue Cursor Iteration & 5 Track Inserts          3.40 ms           |
|   sp_advance_queue Promotion & Status Updates               1.20 ms           |
|   Audit Trail Write (vibe_shift_log)                        0.55 ms           |
|   ----------------------------------------------------   ------------         |
|   TOTAL AUTOPILOT PIVOT TRANSACTION TIME                    9.59 ms           |
+-------------------------------------------------------------------------------+
```

The entire pivot operation—encompassing graph traversal, queue truncation, cursor fetching, queue population, track advancement, and audit logging—executes inside PostgreSQL in **under 10 milliseconds**.

Transmitted over WebSockets / HTTP polling, the client interface reflects the stylistic pivot in less than 50 milliseconds end-to-end, vastly outperforming human auditory observation latency (typically 15 to 60 seconds).

---

8. ARCHITECTURAL EVOLUTION, REFINEMENTS, AND ENGINEERING INSIGHTS

Throughout the engineering lifecycle of the project, several critical design challenges emerged, prompting substantive refactorings that provide valuable lessons for database systems engineering:

8.1 Resolution of External API Deprecations

The initial specification relied upon the real-time Spotify Web API for streaming track audio and fetching live audio features. On November 27, 2024, Spotify deprecated the `/v1/audio-features` endpoint for new developer accounts and permanently ceased populating `preview_url` fields.

- **Engineering Pivot:** The architecture was redesigned to decouple completely from third-party cloud streaming. The ML feature extractor was redirected to offline standardized Kaggle audio feature exports (e.g., 114k tracks), and the frontend client was upgraded with a procedural Web Audio API synthesizer that generates tempo-matched acoustic rhythms shaped by the active vibe cluster's energy envelope.

8.2 Eliminating Autopilot Cascades

In early trigger drafts, vibe scores were computed as an unconstrained sum across the entire history of the stage. Once a stage crossed below the -50–50 threshold, **every single subsequent vote—even positive votes—re-tripped the trigger**, causing the queue to infinitely thrash and rewrite on every click.

- **Engineering Pivot:** As detailed in Section 5.1, the query was re-engineered to compute scores strictly within a dynamic temporal window ($\Delta t \leq$ window_secondsΔt ≤ window_seconds) bounded below by the stage's `last_pivot_at` timestamp. Furthermore, an enforced `cooldown_seconds` (20 s) guard rail was added to guarantee set stability immediately following a vibe shift.

8.3 Clarification of Two-Phase Locking Semantics

Early slide documentation stated that locks are released "immediately after the mathematical calculation." In DBMS theory, releasing locks prior to transaction termination violates Two-Phase Locking, reducing the protocol to basic short-term locking and exposing the system to dirty reads and non-serializable interleavings.

- **Engineering Pivot:** The transaction boundary was rigorously aligned with Strict 2PL. The exclusive lock acquired in `fn_vote_before` is maintained continuously through trigger execution and procedure execution, releasing strictly upon PostgreSQL transaction commit.

---

9. FUTURE WORK & EXTENSIONS

1. **Spatial Multi-Stage Acoustic Bleed Optimization:** Expanding the database engine to model multi-stage spatial topography. Using spatial extensions (PostGIS), the database could calculate acoustic sound bleed

between adjacent physical stages and coordinate dual-stage queue rewrites to prevent clashing BPMs across physical festival grounds.

2. **Reinforcement Learning on Adjacency Weights:** Replacing static centroid Euclidean distance with an active reinforcement learning model (e.g., Q-learning). Transition edges in `cluster_adjacency` could dynamically adjust weights based on crowd retention rates following historical pivots.
3. **Distributed Multi-Master Sharding:** For multi-million-user global virtual events, evaluating Citus / distributed PostgreSQL clustering to partition stage voting tables across independent shard nodes while maintaining raft-coordinated state consistency.

---

10. CONCLUSION

The Crowdsourced Festival Stage Optimizer successfully demonstrates that the relational database management system can transcend its traditional role as a passive persistence layer. By embracing a **"Thick-Database" Architecture**, the system centralizes complex event telemetry, statistical time-window aggregation, procedural queue restructuring, and topological graph traversal entirely within PostgreSQL.

The resulting engine achieves sub-10ms autonomous queue pivot latency, guarantees complete immunity to lost updates under massive concurrent load via Strict Two-Phase Locking, and enforces structural integrity through rigorous 3NF relational normalization. This project provides a robust, reproducible template for engineering high-contention, autonomous cyber-physical event systems driven by crowdsourced data.

---

11. REFERENCES & BIBLIOGRAPHIC CITATIONS

1. Bernstein, P. A., Hadzilacos, V., & Goodman, N. (1987). *Concurrency Control and Recovery in Database Systems*. Addison-Wesley.
2. Codd, E. F. (1970). "A Relational Model of Data for Large Shared Data Banks." *Communications of the ACM*, 13(6), 377–387.
3. Gray, J., & Reuter, A. (1992). *Transaction Processing: Concepts and Techniques*. Morgan Kaufmann.
4. MacQueen, J. (1967). "Some Methods for Classification and Analysis of Multivariate Observations." *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*, 1, 281–297.
5. Stonebraker, M., Hanson, E. N., & Potamianos, S. (1988). "The POSTGRES Rule System." *IEEE Transactions on Software Engineering*, 14(7), 897–907.
6. The PostgreSQL Global Development Group. (2024). *PostgreSQL 16 Documentation: Chapter 13. Concurrency Control & Chapter 43. PL/pgSQL*.
7. Tzanetakis, G., & Cook, P. (2002). "Musical Genre Classification of Audio Signals." *IEEE Transactions on Speech and Audio Processing*, 10(5), 293–302.
